

BITÁCORA DE PROYECTO

Inventory App — Sistema de Gestión de Inventario

Autor:	Mario Borregán Barral
Tutor:	Damián Sauldea
Titulación:	Desarrollo de Aplicaciones Multiplataforma (DAM)
Fecha:	Mayo 2026

1. Introducción

Este documento recoge el desarrollo del Trabajo de Fin de Grado correspondiente al ciclo formativo de Desarrollo de Aplicaciones Multiplataforma. El proyecto consiste en el desarrollo de una aplicación completa de gestión de inventario orientada a pequeñas empresas y tiendas, que incluye un backend con API REST, una aplicación web y una aplicación móvil.

El objetivo principal fue crear una solución funcional y real que cubra las necesidades básicas de control de stock, gestión de productos, categorías y movimientos de inventario, con un sistema de autenticación por roles y una interfaz intuitiva tanto en web como en móvil.

2. Tecnologías Utilizadas

Capa	Tecnología	Versión
Backend	FastAPI + Python	Python 3.11
Base de datos	SQLite + SQLAlchemy	—
Autenticación	JWT (JSON Web Tokens)	—
Frontend Web	React + Vite + Axios	React 18
App Móvil	Flutter + Dart	Flutter 3.41
Documentación API	Swagger / OpenAPI	OAS 3.1

3. Arquitectura del Sistema

El sistema sigue una arquitectura de tres capas bien diferenciadas. El backend actúa como núcleo central exponiendo una API REST a la que se conectan tanto la aplicación web como la aplicación móvil.

3.1 Backend (FastAPI)

El backend está desarrollado con FastAPI y sigue una estructura por capas: modelos SQLAlchemy para la base de datos, schemas Pydantic para la validación de datos, routers para los endpoints y un módulo de seguridad para la gestión de JWT.

La estructura de carpetas del backend es la siguiente:

- app/models/ — Modelos de base de datos (User, Product, Category, Movement)
- app/schemas/ — Validación de datos con Pydantic
- app/routers/ — Endpoints de la API (auth, products, categories, movements)
- app/core/ — Configuración y seguridad (JWT, hashing de contraseñas)
- app/database.py — Conexión y sesión de base de datos

3.2 Base de Datos

Se utiliza SQLite para el entorno de desarrollo por su simplicidad, aunque el código está preparado para migrar a PostgreSQL en producción simplemente cambiando la cadena de conexión en el archivo .env.

3.3 Frontend Web (React)

La aplicación web está desarrollada con React y Vite. Se comunica con el backend mediante Axios y gestiona la autenticación almacenando el token JWT en localStorage.

3.4 Aplicación Móvil (Flutter)

La aplicación móvil está desarrollada con Flutter, lo que permite compilarla tanto para Android como para iOS desde el mismo código. Se comunica con el backend mediante peticiones HTTP y almacena el token JWT usando SharedPreferences.

4. Funcionalidades Implementadas

4.1 Sistema de Autenticación

El sistema incluye un módulo completo de autenticación con registro de usuarios, login con generación de token JWT y protección de endpoints según el rol del usuario. Los roles disponibles son: admin, manager y employee.

- Registro de nuevos usuarios con email, nombre, contraseña y rol
- Login con validación de credenciales y generación de JWT
- Token con expiración configurable mediante variables de entorno
- Contraseñas almacenadas con hash bcrypt
- Cierre de sesión con eliminación del token local

4.2 Gestión de Productos

El módulo de productos es el núcleo del sistema. Permite gestionar todo el catálogo de la tienda o empresa.

- Listado completo de productos con nombre, SKU, stock y precio
- Creación de nuevos productos con todos sus campos (nombre, SKU, descripción, categoría, precio, coste, stock, stock mínimo, unidad, proveedor)
- Edición de productos existentes
- Eliminación de productos con confirmación
- Búsqueda en tiempo real por nombre o SKU
- Filtrado por stock bajo (productos con stock igual o menor al mínimo)
- Vista de detalle completa de cada producto
- Indicador visual de stock bajo en rojo y stock correcto en verde

4.3 Gestión de Categorías

Las categorías permiten organizar los productos de forma lógica y visual.

- Listado de categorías con nombre, descripción y color identificativo
- Creación de categorías con selector de color
- Eliminación de categorías con confirmación

4.4 Gestión de Movimientos de Inventario

Los movimientos registran todos los cambios de stock, lo que permite tener un historial completo de lo que ha pasado con el inventario. Cada movimiento actualiza automáticamente el stock del producto correspondiente.

- Tres tipos de movimiento: entrada (aumenta stock), salida (reduce stock) y ajuste (establece stock exacto)
- Registro de motivo, notas adicionales y referencia (número de factura, orden, etc.)
- Snapshot del nombre del producto en el momento del movimiento
- Validación de stock suficiente antes de registrar una salida
- Historial completo con fecha y hora de cada movimiento
- Filtrado por tipo de movimiento (entrada, salida, ajuste)

4.5 Dashboard / Pantalla de Inicio

La pantalla principal ofrece un resumen visual del estado del inventario de un vistazo.

- Tarjetas de resumen con total de productos, categorías, movimientos y alertas de stock bajo
- Lista de últimos movimientos registrados
- Lista de productos con stock bajo destacados en rojo
- Actualización de datos con pull to refresh

4.6 API REST y Documentación

El backend expone una API REST completamente documentada y accesible mediante Swagger UI en /docs.

- Endpoints CRUD completos para productos, categorías y movimientos
- Endpoint especial GET /products/low-stock para productos con stock bajo
- Documentación interactiva con Swagger / OpenAPI accesible en <http://127.0.0.1:8000/docs>
- Validación automática de datos con Pydantic
- Manejo de errores con códigos HTTP correctos (400, 401, 404, 500)
- CORS configurado para permitir conexiones desde el frontend web y la app móvil

5. Mejoras Futuras y Líneas de Trabajo

El proyecto tiene una base sólida sobre la que se pueden implementar muchas mejoras. A continuación se listan las que se consideran más relevantes para hacer el sistema aún más completo y profesional.

5.1 Mejoras Técnicas

- Migración de SQLite a PostgreSQL para entornos de producción
- Implementación de Alembic para migraciones de base de datos versionadas
- Tests automatizados del backend con pytest
- Despliegue del backend en AWS (EC2 o Lambda) con base de datos en RDS
- Configuración de CI/CD con GitHub Actions
- Soporte para múltiples tiendas o almacenes en el mismo sistema

5.2 Nuevas Funcionalidades

- Autenticación con Google OAuth para facilitar el registro e inicio de sesión
- Sistema de notificaciones push cuando el stock baja del mínimo
- Generación de informes y exportación a PDF o Excel
- Gestión de proveedores como entidad independiente con su propio CRUD
- Historial de precios y costes de productos
- Códigos de barras o QR para identificar productos rápidamente
- Modo offline en la app móvil con sincronización cuando vuelve la conexión
- Panel de administración para gestión de usuarios y roles

5.3 Integración de Inteligencia Artificial

Una de las líneas de mejora más interesantes es la incorporación de funcionalidades basadas en IA, algo que cada vez es más demandado en aplicaciones empresariales.

- Predicción de demanda basada en el historial de movimientos para anticipar cuándo reponer stock
- Detección de anomalías en los movimientos (salidas inusuales que podrían indicar errores o robos)
- Recomendaciones automáticas de cantidad de pedido a proveedores
- Clasificación automática de productos en categorías usando procesamiento de lenguaje natural
- Chatbot integrado para consultar el estado del inventario en lenguaje natural

5.4 Mejoras de Diseño y Experiencia de Usuario

- Diseño responsivo completo para todos los tamaños de pantalla
- Tema oscuro / claro configurable por el usuario
- Onboarding para nuevos usuarios con tutorial interactivo
- Gráficas y visualizaciones de datos en el dashboard
- Animaciones y transiciones más fluidas en la app móvil

6. Conclusiones

El desarrollo de este proyecto ha sido una experiencia muy completa que ha permitido aplicar de forma práctica muchos de los conocimientos adquiridos durante el ciclo formativo. Se ha conseguido desarrollar una aplicación funcional de principio a fin, desde el diseño de la base de datos hasta las interfaces de usuario.

Lo más valorable del proyecto es que sigue una arquitectura real utilizada en el mundo profesional, con separación de responsabilidades, autenticación segura y documentación automática. Además, al estar construido sobre tecnologías modernas y ampliamente utilizadas como FastAPI, React y Flutter, el código es mantenible y escalable para futuras mejoras.

Aunque queda camino por recorrer en cuanto a funcionalidades avanzadas y despliegue en producción, la base desarrollada es sólida y representa un punto de partida muy bueno para un sistema de gestión de inventario real.